Mini-Project / Technical Demonstrator

AI Interactive Learning Demonstrator

A voice-driven AI tutor that builds interactive learning simulations on demand

Project Report • Concept, Architecture & Technology
Prepared as reference documentation for report authoring

1. Executive Summary

The AI Interactive Learning Demonstrator is a web application that lets a student **talk to an AI tutor about something they don't understand**, and — instead of merely explaining it — the system **generates a custom, interactive simulation** tailored to that exact confusion, then renders it live in the browser for the student to experiment with.

The core innovation is the pipeline that converts a natural, roughly two-minute spoken conversation into a structured *Learning Brief*, and then uses an AI coding agent to write a self-contained HTML / CSS / JavaScript simulation targeted at the student's specific misunderstanding. The result is served by the backend and displayed inside a security-sandboxed frame.

> **The one-line pitch:** "Talk to an AI tutor, and instead of just explaining the concept, it builds an interactive experiment for you."

What makes it different

A typical AI tutor answers questions with text. This project answers with a *manipulable demonstration* — sliders, animations, live values, draggable points — because many concepts (Newton's laws, derivatives, sorting algorithms) are understood far better by interacting with them than by reading about them.

2. The Idea & Concept

The problem

When a student is stuck, a generic explanation often misses the actual point of confusion. Understanding is also highly individual — one student needs to *see* motion, another needs the equation. Static answers can't adapt to that.

The solution — a generative learning loop

The system discovers the student's precise learning need through conversation, distills it into a compact brief, and generates a fresh interactive artifact for that need. Every student gets a bespoke demonstration rather than a canned lesson.

End-to-end user journey

```
🎙 Talk → 🧠 Understand → ⚙ Build → ✓ Ready → 🪄 Experiment → 💬 Ask follow-up
```

Scope: a convincing demonstrator

This is deliberately a **mini-project / demonstrator**, not a production platform. Engineering effort is concentrated on the critical path (voice → brief → generation → render), and explicitly excludes authentication, databases, cloud storage, multi-tenancy, payments, and heavy sandbox infrastructure.

3. How It Works — The Pipeline

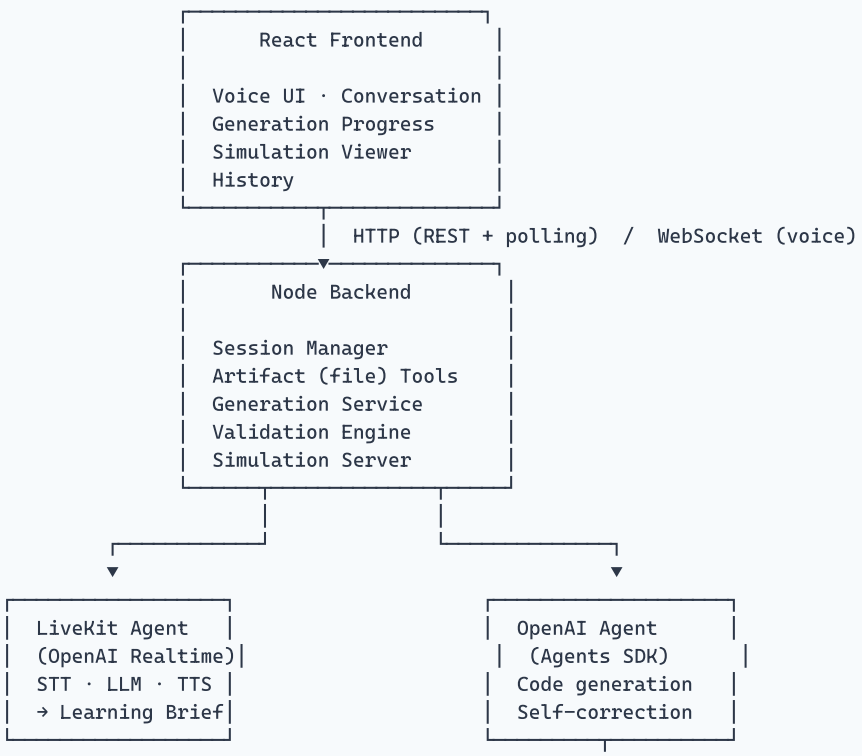
The entire product is one data pipeline. Each stage has a single, well-defined responsibility:

```
Student
   │  speaks naturally (~2 minutes)
   ▼
LiveKit Voice Agent  ──────────  listens, asks follow-ups, identifies the confusion
   │
   ▼
Learning Brief (JSON)  ─────────  compact structured summary of the learning need
   │
   ▼
OpenAI Generation Agent  ──────  writes index.html + style.css + app.js via file tools
   │
   ▼
Validation + Self-Correction ──  checks files; feeds errors back; up to 3 attempts
   │
   ▼
Static Simulation Server  ──────  serves the generated files per session
   │
   ▼
Sandboxed <iframe>  ──────────  student interacts with the live simulation
   │
   ▼
Follow-up conversation  ────────  student keeps asking; can regenerate
```

Why the Learning Brief matters

The brief is the contract between the voice side and the generation side. Because it is plain structured JSON, the two halves are **decoupled**: the fragile, external voice layer can fail or be swapped without affecting the generation pipeline, and a brief can even be typed in by hand as a fallback.

4. System Architecture





All secrets (OpenAI key, LiveKit secret) live only on the backend. The browser never receives them; for voice it receives only a short-lived LiveKit room token minted server-side.

5. Technology Stack

A key simplification: using **OpenAI for both voice and generation** means the whole system needs only one AI provider.

Layer	Technology	Why
Frontend	React + Vite + TypeScript + Tailwind CSS + Framer Motion	Fast dev loop, typed UI, utility styling, smooth microinteractions
Backend	Node.js + TypeScript + Fastify	Lightweight, high-performance HTTP server with first-class TypeScript and static-file support
Voice	LiveKit + LiveKit Agents + OpenAI Realtime API	Real-time audio transport; Realtime does speech-to-text, reasoning and text-to-speech in one model
AI Generation	OpenAI Agents SDK (`@openai/agents`) + Zod	Agent loop with typed, schema-validated tool calls for safe file operations
Generated output	HTML, CSS, vanilla JavaScript, SVG, Canvas	Self-contained, no build step, no external dependencies — runs anywhere
Storage	Filesystem (one folder per session)	No database needed for a demonstrator; trivially inspectable
Runtime	tsx (TypeScript execution)	Run TypeScript directly, no separate build step during development
Validation	Node `vm` module + regex scanning	In-process JS syntax check and security scan without executing untrusted code

Why OpenAI Realtime for voice

- One model handles STT + reasoning + TTS → one API, one key, one websocket.
- Supports structured *tool/function calls*, so the agent emits the Learning Brief as validated JSON instead of free-form text that must be parsed.
- No separate speech vendors (e.g. Deepgram/ElevenLabs) to configure.

6. Repository Structure

```
AIminiProject/
├── apps/
│   ├── web/                    React + Vite frontend (Phase 5)
│   │   └── src/
│   │       ├── pages/          Home · Session · History
│   │       ├── components/     VoicePanel · SimulationViewer · GenerationProgress
│   │       ├── hooks/          useSession · useGenerationStatus · useLiveKit
│   │       └── lib/            API client
│   └── server/                 Fastify backend (built, Phases 1–4)
│       └── src/
│           ├── routes/         sessions · generation · simulations
│           ├── sessions/       session manager + learning-brief storage
│           ├── agents/         generation agent · tools · system prompt · mock
│           ├── artifacts/      path guard (security) + file tools
│           ├── validation/     validator + self-correction loop
│           ├── services/       generation orchestration + status
│           └── server.ts
├── voice/                      LiveKit Agents worker (Phase 6)
├── sessions/                   runtime session data (git-ignored)
```

```
├── .env.example
└── package.json                npm workspaces monorepo
```

7. Core Components & Implementation

7.1 Session Manager Built

Every interaction is a *session* stored as a folder on disk. The manager creates sessions with unique IDs (`session_<6 hex>`), writes `session.json` metadata, and lists history by scanning the sessions directory — no database required.

7.2 Path-Safe Artifact Tools Built

The generation agent manipulates files through five tools — `list_files` , `read_file` , `write_file` , `update_file` , `delete_file` — but every path is forced through a **path guard**. This is the project's central security boundary.

- All paths resolve against `sessions/<id>/simulation/` and may never escape it.
- Rejects `..` traversal, absolute paths, Windows drive paths, null bytes, and symlink escapes.
- The session ID is bound in a closure — it is *not* a model-supplied parameter — so the AI cannot choose or tamper with which session it writes to.

7.3 Generation Agent Built

Built on the OpenAI Agents SDK. Given a Learning Brief, it is instructed (via a strong "Educational Simulation Engineer" system prompt) to create an **interactive demonstration** — not a page of text — using only HTML, CSS, vanilla JS, SVG and Canvas, with no external libraries or network calls. It writes the files through the guarded tools, then inspects and fixes its own work before finishing.

7.4 Validation + Self-Correction Built

After generation, the files are validated:

- **Existence:** `index.html` , `style.css` , `app.js` must all exist.
- **References:** the HTML must link the CSS and the JS.
- **Syntax:** `app.js` is compile-checked in-process via Node's `vm` module (no execution).
- **Security scan:** flags `fetch` , `XMLHttpRequest` , `WebSocket` , `eval` , `new Function` , and any external script/stylesheet.

If validation fails, the errors are fed back to the agent, which reads and repairs the files. This loops a maximum of **3 attempts**; if it still fails, the session is marked as an error and the user is shown a friendly retry message.

7.5 Static Simulation Server Built

Serves `GET /simulations/:sessionId/*` directly from the session's `simulation/` folder, through the same path guard, with correct content types. The real filesystem path is never exposed to the client.

7.6 Rendering — Sandboxed iframe Phase 5

The React app loads the simulation URL inside `<iframe sandbox="allow-scripts">` . Generated code is treated as untrusted: it runs isolated, cannot reach the parent app, cannot make network requests, and is never injected via `dangerouslySetInnerHTML` .

8. Security Design

Because the system runs AI-generated code, security is layered deliberately:

Concern	Mitigation
Agent escaping the session folder	Path guard: rejects traversal, absolute paths, symlinks; confined to `simulation/`
Agent choosing the wrong session	Session ID bound in closure, never a tool parameter
Malicious generated JavaScript	Validation security scan + sandboxed iframe (`allow-scripts` only)
Generated code calling the network	Scan rejects fetch/XHR/WebSocket; iframe sandbox blocks same-origin access

Secret leakage to the browser	OpenAI key & LiveKit secret are server-only; frontend gets a short-lived room token
Exposing server internals	Errors logged server-side; students see friendly messages, never stack traces

9. Data Contracts

Session metadata — `session.json`

```
{
  "id": "session_8f2c91",
  "createdAt": "2026-10-08T14:30:00.000Z",
  "status": "ready",
  "subject": "Physics",
  "topic": "Newton's Second Law",
  "simulation": { "status": "ready", "entryPoint": "simulation/index.html" }
}
```

Status lifecycle: created → conversation → understanding → generating → validating → ready / error

Learning Brief — `learning-brief.json`

```
{
  "subject": "Physics",
  "topic": "Newton's Second Law",
  "studentLevel": "beginner",
  "studentAlreadyUnderstands": ["force", "mass"],
  "confusion": "How changing mass or force affects acceleration.",
  "learningGoal": "Understand the relationship between force, mass and acceleration.",
  "preferredDemonstration": "Interactive visual physics simulation",
  "requiredInteractions": ["change force", "change mass", "observe acceleration"]
}
```

Generation status (polled during generation)

```
{ "status": "generating", "step": "creating_files",
  "message": "Building interactive simulation ... " }
```

Steps: understanding → planning → creating_files → validating → fixing → ready / error

10. Backend API Reference

Method & Path	Purpose
`POST /api/sessions`	Create a new session → returns `{ sessionId }`
`GET /api/sessions`	List session history (newest first)
`GET /api/sessions/:id`	Get session metadata
`POST /api/sessions/:id/learning-brief`	Store / update the Learning Brief
`POST /api/sessions/:id/generate`	Start async generation (returns immediately)
`GET /api/sessions/:id/status`	Poll generation progress
`GET /api/sessions/:id/simulation`	Simulation metadata `{ ready, url }`
`GET /simulations/:id/*`	Serve generated simulation files (path-guarded)
`POST /api/livekit/token` Phase 6	Mint short-lived LiveKit room token

> **Async generation:** `POST /generate` returns right away; the frontend polls `/status` every 1–2 seconds. No WebSocket is introduced just for generation progress — LiveKit already handles the real-time voice channel.

11. Example Generated Simulation

For a Newton's Second Law brief, the agent produces a draggable physics demo. The student changes *Force* and *Mass* with sliders and immediately sees acceleration and the box's motion respond — reinforcing `F = m × a` through experimentation rather than text.

```
Force  [──────────●──────]  40 N
Mass   [────●────────────]  10 kg

     ←── ┌──────────┐
         │   BOX    │ ──────────────→   (animated)
         └──────────┘

Force: 40 N   ·   Mass: 10 kg   ·   Acceleration: 4.0 m/s²

Increase force → acceleration ↑ · box speeds up
Increase mass  → acceleration ↓ · box slows down
```

Other subjects map to other visual forms: Mathematics → SVG graphs with draggable points; Chemistry → animated particle diagrams; Computer Science → animated sorting / data-structure steps.

12. Implementation Status & Roadmap

Development follows a critical-path-first order, so the core value is proven before UI and voice polish.

Phase	Deliverable	Status
1	Monorepo scaffold + session CRUD (filesystem storage)	Complete
2	Path-safe artifact file tools (security boundary)	Complete
3	Generation agent + static simulation server	Complete
4	Full validation + 3-attempt self-correction loop	Complete
5	React frontend: Home / Session / History, sandboxed iframe, generation-progress UI, typed-brief fallback	Remaining
6	Voice: LiveKit + OpenAI Realtime, token endpoint, 2-minute discovery → Learning Brief	Remaining
7	Polish: history reopen, accessibility, microinteractions, README	Remaining

Verification so far: the backend passes a 17-test automated suite (path-guard security, validation rules, and the self-correction loop) plus an end-to-end pipeline check that drives a brief through generation to a served, validated simulation.

> **Design choice — voice last:** voice is the most fragile, externally-dependent piece, so it is built last and a typed-brief fallback is kept permanently. A LiveKit outage can never block the core demonstration.

13. How to Run (current backend)

Prerequisites	Node.js 22+, npm 10+
Install	`npm install`
Run backend	`npm run dev:server` → serves on `http://localhost:3001`
Run tests	`npm run test:server` (17 tests) · `npm run typecheck`
Config	Copy `.env.example` → `.env`; set `OPENAI_API_KEY` for live generation

> **Offline demo mode:** with no `OPENAI_API_KEY`, generation falls back to a built-in interactive Newton's Second Law simulation, so the full pipeline (generate → validate → serve → render) can be demonstrated without any API calls. Adding a key automatically switches to the real AI agent — no code change.

14. Conclusion

The AI Interactive Learning Demonstrator shows a compelling idea in working form: an AI tutor that responds to a student's confusion by **building a custom interactive experiment** rather than reciting an explanation. The backend pipeline — session management, a security-hardened AI code generator, and a self-correcting validation loop — is implemented and tested. What remains is the visible surface (frontend) and the voice front door, both of which plug into the already-proven core.
